Análisis léxico

Santiago Javier Proaño Yépez

<2026-10-19 lun>

Contents

1 Análisis léxico **1**
1.1 Lexema 2
1.2 Patrón 2
1.2.1 Patrón identificador 2
1.3 Token 3
1.4 Ejemplo ATTACH 3
1.5 Errores léxicos ATTACH 4
1.6 Manejo de buffers 5

1 Análisis léxico

También conocido como scanner, es el bloque funcional que hace de interfaz para el programador y el compilador, primera fase de la compilación. Básicamente, va de arriba a abajo y de izquierda a derecha recorriendo el programa fuente carácter por carácter, esa es la entrada, luego se reconoce el patrón al que cae y por ultimo del analizador salen los tokens o también llamados componentes léxicos.

En esta fase se ignora y eliminan: comentarios, espacios en blanco, tabuladores, saltos de linea entre otros, dejando al texto plano listo para el analizador sintáctico. Además de que se notifican los errores léxicos y ayuda a corregirlos.

- Modo pánico, básicamente elimina los caracteres, arregla la entrada o recupera palabras reservadas para encontrar un token bien formado.

Por otro lado mientras sacan los tokens estos son guardados en la tabla se símbolos.

Una de sus funciones es comprobar que estén bien escritos los símbolos de un lenguaje de programación:

- Identificadores: epn25, jisisc2026, x ...
- Números: 1, 2, -3 ...
- Palabras reservadas: if, while, for, ...
- Operadores aritméticos: +, -, * ...
- Operadores racionales: >, <, = ...
- Símbolos de agrupación: (), {}, []

  Esta fase tiene 3 conceptos que no deben confundirse.

1.1 Lexema

Es una secuencia de caracteres, delimitada usualmente por un vacío. Usa un puntero para recorrer los caracteres y cuando se encuentra con un vacío se lo pone como lexema por ejemplo

```
epn25_
^    ^
|    |
```

el lexema sería epn25

1.2 Patrón

Es la regla con la que se analiza al lexema y con la cual se decide a que token pertenece.

1.2.1 Patrón identificador

Define lo que es y no es un identificador

$$\begin{aligned} & l(l|d)^* \\ l \rightarrow |a|b|\ldots|z|A|B|\ldots|Z| & \\ d \rightarrow |0|\ldots|9| & \end{aligned}$$

1.3 Token

Es la categoría en donde cae el lexema luego de ser analizado su patrón. Por ejemplo ID, ASIG para el =, NUM para un número etc.

1.4 Ejemplo ATTACH

En el ejemplo se hace una correspondencia entre el lexema y un número que identifica a token.

Token	Identificación del token
ID	27
CTE	28
IF	59
THEN	60
ELSE	61
+	70
/	73
>=	80
:=	85

Teniendo el código fuente

```
if Plazo >= 30
```

```
then Tasa = Base + Recargo / 100
else Tasa = Base
```

Se reconoce cada lexema

```
[if] [Plazo] [>=] [30]
 [then] [Tasa] [=] [Base] [+] [Recargo] [/] [100]
 [else] [Tasa] [=] [Base]
```

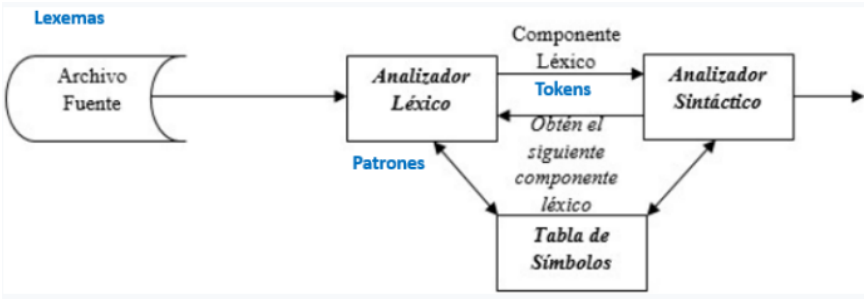
Entonces se transforma a números, si un token puede tener varios lexemas se pasa al analizador sintáctico el token-lexema. por ejemplo

```
[59] [27, 'Plazo'] [80] [28, '30']
 [60] [27, 'Tasa'] [85] [27, 'Base'] [70] [27, 'Recargo'] [73] [28, '100']
 [61] [27, 'Tasa'] [85] [27, 'Base']
```

luego se juntan todos los números y se pasan los tokens sin espacio, aparte de hacer la tabla se simbolos

```
IF ID >= CTE THEN ID = ID + ID / CTE ELSE ID = ID
```

Por lo tanto el esquema de esta fase seria asi:



1.5 Errores léxicos ATTACH

El analizador rechaza todos los caracteres o combinaciones ilegales, es decir todo lo que no esta reconocido en el alfabeto o que son caracteres mal puestos. Por ejemplo:

- Las tildes (“é”, “ají”, etc)
- Incoincidencias (:::, », etc) básicamente patrones que no coincide con los tokens posibles.

Entonces detectado el error el analizador deberá mostrar un mensaje de error de forma correcta, mostrando la linea, columna y el error en si. por ejemplo:

```
int número = 1 ;
     ^
ERROR 124: línea 85, columna 6, carácter no válido
```

1.6 Manejo de buffers

Este scanner debe leer el archivo carácter por carácter, sin embargo es muy lento leer el programa del disco duro a diferencia del CPU. Sin embargo el truco esta en que el costo del sistema operativo para leer es lo mismo si es 1 o 4096 bites.

Para que se entienda como analogía, si quieres mover 10 000 cajas de un lado a otro es más eficiente llevar en una carretilla 4096 cajas en vez de 1 a 1. Por lo tanto el buffer vendira a ser esa carretilla.